

# Нейронные сети классификации изображений

Классификация изображений это базовая задача обработки изображений, наиболее изученная, с наибольшим числом разнообразных нейронных сетей, которые ее решают.

Многие другие задачи обработки изображений используют для своего решения части нейронных сетей полученные именно для классификации.

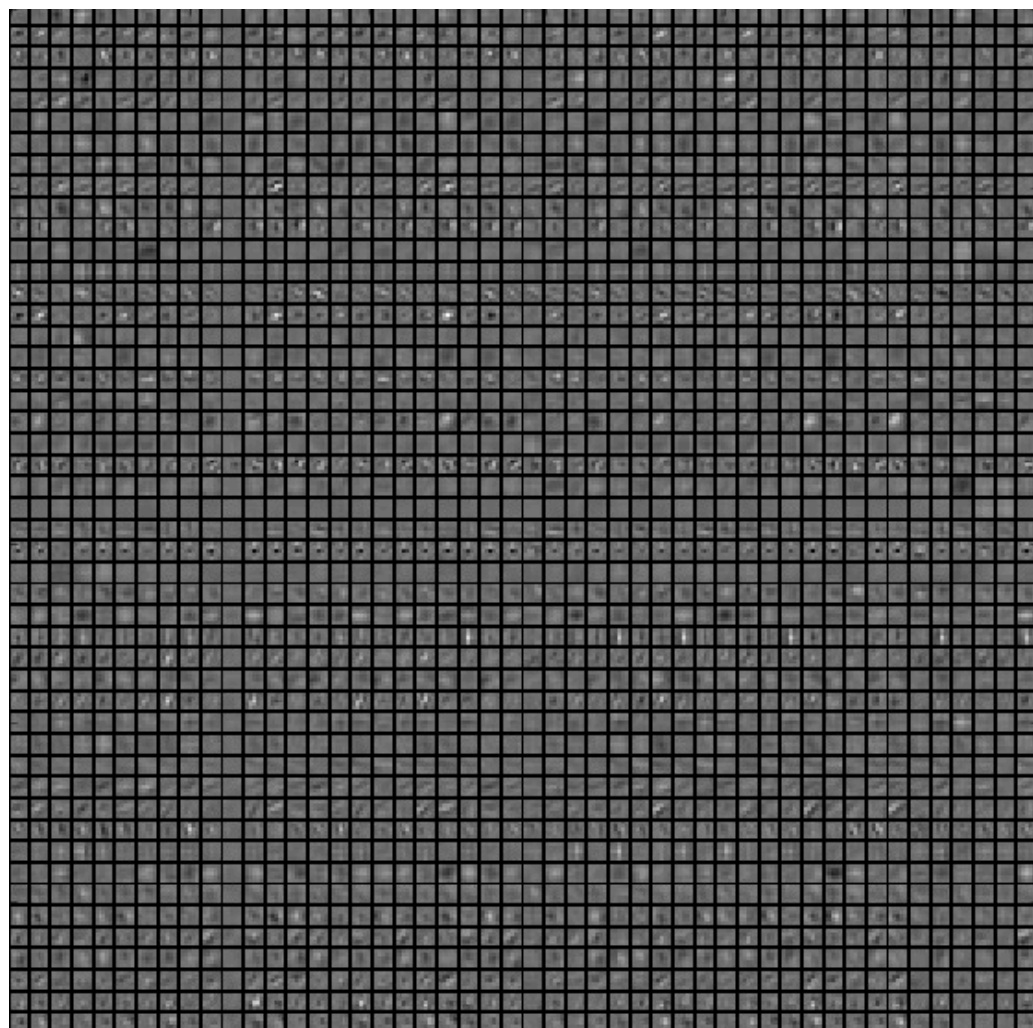
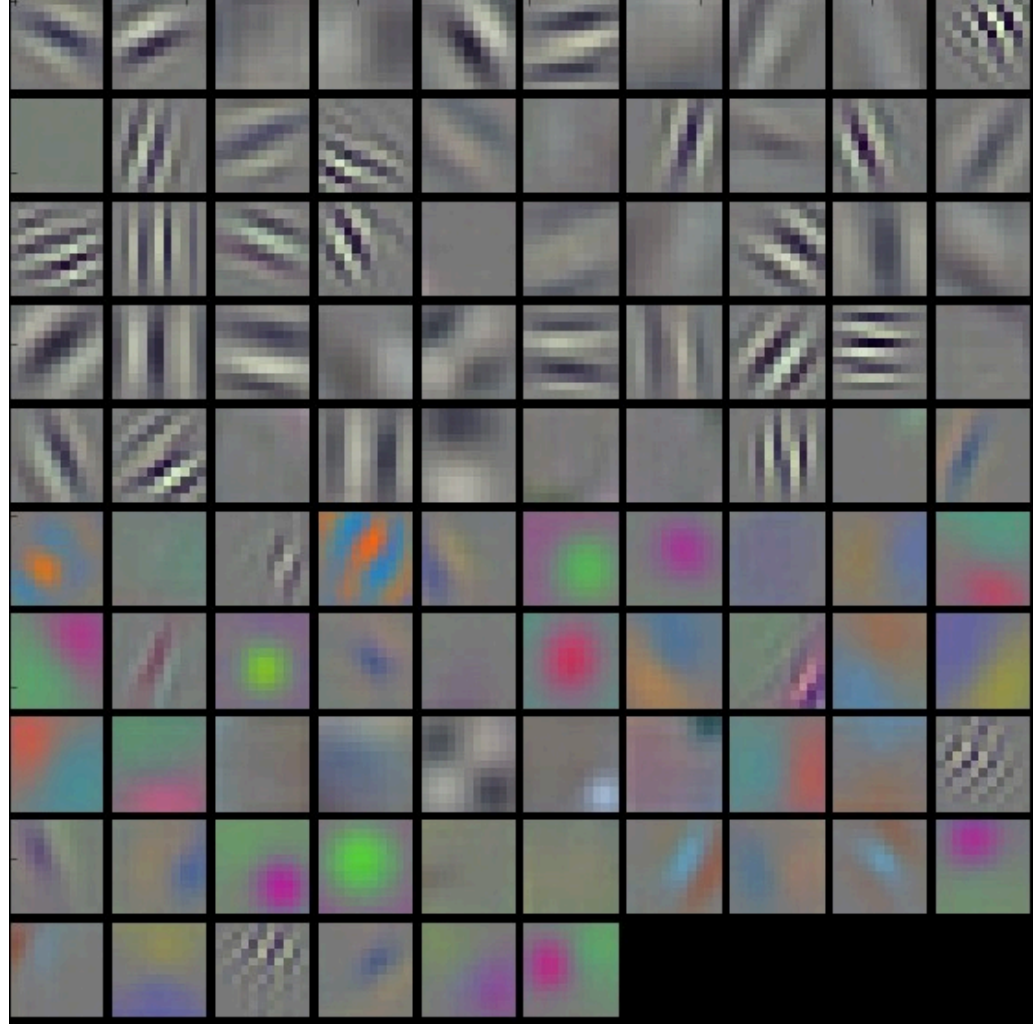
Число различных архитектур для классификации изображений велико, десятки, если не сотни. Познакомимся только с базовыми подходами к созданию таких сетей.

Простые, но все еще достаточно эффективные сети - это слоистые сети с последовательным подключением слоев. Первые слои таких сетей это сверточные слои, последние - полносвязные, причем последний слой имеет столько нейронов, сколько классов распознается и, как правило, сопровождается функцией активации softmax. Представители: **LeNet**, **AlexNet**.

Более сложные сети, придуманные для упрощения построения глубоких сетей, это блочные сети, в которых используются последовательно подключенные блоки слоев. Сами блоки регулярны, строятся по одному принципу, например чередованием сверточных слоев, активации и субдискретизации. Блочность упрощает разработку сетей и их описание. Представители: **VGG**

Трудно решить заранее, какие именно должны быть параметры сверток, например, что лучше, свертка с ядром  $11 * 11$  или  $3 * 3$ ? А может вообще использовать не свертку, а субдискретизацию? Все они могут быть полезны! Чтобы не ломать голову такими экспериментами решили делать разветвленные архитектуры, в которых к одному входу применяется одновременно и параллельно несколько разных сверток, результаты которых затем объединяются. Так получились блочно-последовательные сети с разветвленными блоками. Представители **NiN**, **GoogleNet**.

В сетях с последовательным подключением слоев для классификации изображений наблюдается иерархия слоев. Слои с небольшим номером реагируют (фильтруют) простые формы - кусочки линий, цветные пятна, текстуры. Слои же с большим номером реагируют уже на более абстрактные формы, например что-то похожее на лицо человека (если обучалось на изображениях лиц). Часто бывает так, что для классификации важны не только абстрактные признаки, но и детальные. В этом случае в последовательные сети добавляют связи, которые минуют некоторые слои, передавая детальную информацию на более дальние слои. Представители: **ResNet**, **DenseNet**. На рис. показаны фильтры для первого и второго слоев AlexNet.

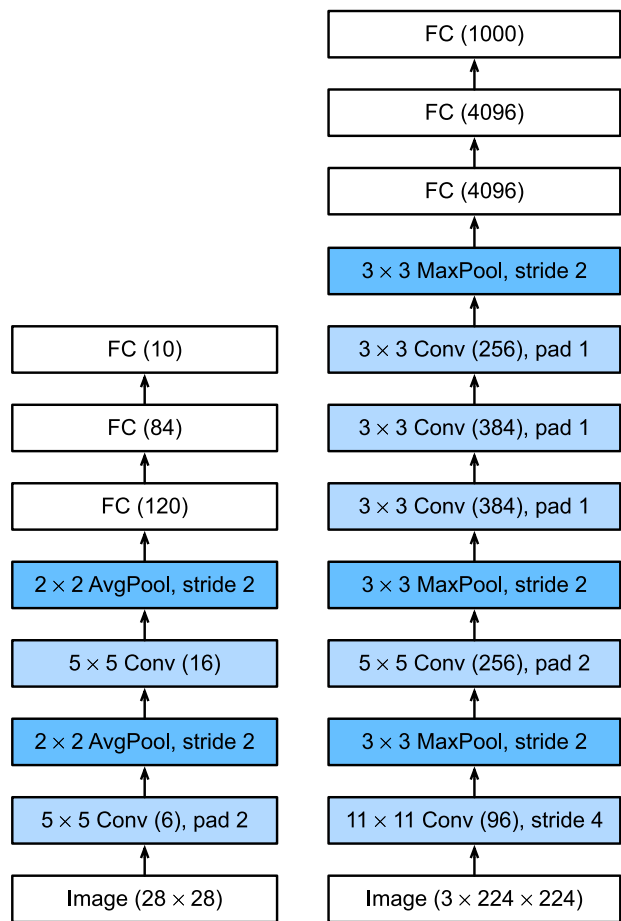


Далее ознакомимся с некоторыми из них более детально. Мы будем пользоваться замечательным курсом [Dive into Deep Learning](#) и попробуем решить задачу классификации изображений с помощью таких сетей.

Для классификации будет использован набор данных FashionMNIST, в котором представлено 10 классов изображений одежды.

## 1 Сеть AlexNet

Сеть AlexNet в свое время победила в соревновании по классификации изображений "ImageNet Large Scale Visual Recognition Challenge 2012" и состоит из 8 сверточных слоев нейронов, двух полносвязных скрытых слоев по 4096 нейрона и полносвязного слоя для классификации с 1000 нейронами (по числу классов изображений на соревновании), а также слоев субдискретизации. Использовались функции активации ReLU, как более простые.



## Архитектура

На вход сети поступает цветное трехканальное изображение  $224 \times 224 \times 3$ .

Первый слой из 96 фильтров с ядром  $11 \times 11$  и сдвигом 4, чтобы уменьшить количество признаков на выходе.

Слои субдискретизации используют maxpooling с ядром  $3 \times 3$  и сдвигом 2, т.е. внахлест.

Второй сверточный слой из 256 фильтров использует ядро  $5 \times 5$  с набивкой 2, чтобы сохранить пространственную размерность (это касается и последующих слоев). 3, 4 и 5 сверточные слои из 384, 394 и 256 фильтров используют ядро  $3 \times 3$  с набивкой 1.

Два полносвязных слоя по 4096 нейронов, также с активацией ReLU и последний полносвязный слой классификации с 1000 нейронов. Большая часть параметров находится в этих полносвязных слоях (~ 1 GB). В наших примерах мы сократим эти размерности.

Создадим сеть на подобии AlexNet для нашей задачи. Слои подключаются последовательно, воспользуемся контейнером Sequential.

Для вспомогательных функций загрузки данных, обучения, отображения, установим библиотеку d2l одноименного курса, при необходимости - перезагрузить "тетрадку".

```
# установим вспомогательные функции из курса d2l
!pip -q install d2l==0.17.0
```

```
→ _____ 83.2/83.2 kB 1.5 MB/s eta 0:00:00
_____ 123.4/123.4 kB 6.7 MB/s eta 0:00:00
_____ 93.5/93.5 kB 5.6 MB/s eta 0:00:00
```

```
import tensorflow as tf #
from d2l import tensorflow as d2l # вспомогательные функции для данных, обучения и отображения

# AlexNet
def net():
    return tf.keras.models.Sequential([ # контейнер
        # большое ядро 11 x 11 для больших объектов, но сдвиг 4, чтобы существенно уменьшить размер выхода
        tf.keras.layers.Conv2D(filters=96, kernel_size=11, strides=4,
                                activation='relu'), # первый сверточный слой
        tf.keras.layers.MaxPool2D(pool_size=3, strides=2), # субдискретизация
        # меньшее ядро 5 x 5, но большее количество фильтров. Набивка, чтобы сохранить пространственные разм
        tf.keras.layers.Conv2D(filters=256, kernel_size=5, padding='same',
                                activation='relu'), # второй сверточный слой
        tf.keras.layers.MaxPool2D(pool_size=3, strides=2), # субдискретизация
        # сверточные слои с малым ядром 3 x 3, с набивкой и еще большим количеством фильтров. Без субдискрет
        tf.keras.layers.Conv2D(filters=384, kernel_size=3, padding='same',
                                activation='relu'),
        tf.keras.layers.Conv2D(filters=384, kernel_size=3, padding='same',
                                activation='relu'),
        # последний сверточный слой с 256 фильтрами, малым ядром 3 x 3.
        tf.keras.layers.Conv2D(filters=256, kernel_size=3, padding='same',
                                activation='relu'),
        tf.keras.layers.MaxPool2D(pool_size=3, strides=2), # субдискретизация
        tf.keras.layers.Flatten(), # слой, который вытягивает входной массив в вектор
        # полносвязный слой
        tf.keras.layers.Dense(4096, activation='relu'),
        tf.keras.layers.Dropout(0.5), # для лучшего обобщения
        # полносвязный слой
        tf.keras.layers.Dense(4096, activation='relu'),
        tf.keras.layers.Dropout(0.5), # для лучшего обобщения
        # последний полносвязный слой с числом нейронов по количеству классов. Для Fashion-MNIST - 10
        tf.keras.layers.Dense(10)
    ])
])
```

Проверим размерности выходов всех слоев. Первое измерение - примеры.

Посчитайте эти размерности самостоятельно, затем объясните вывод.

```
X = tf.random.uniform((1, 224, 224, 3)) # вход (случайно)
for layer in net().layers:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

```
→ Conv2D output shape: (1, 54, 54, 96)
MaxPooling2D output shape: (1, 26, 26, 96)
Conv2D output shape: (1, 26, 26, 256)
MaxPooling2D output shape: (1, 12, 12, 256)
Conv2D output shape: (1, 12, 12, 384)
Conv2D output shape: (1, 12, 12, 384)
Conv2D output shape: (1, 12, 12, 256)
```

```

MaxPooling2D output shape: (1, 5, 5, 256)
Flatten output shape: (1, 6400)
Dense output shape: (1, 4096)
Dropout output shape: (1, 4096)
Dense output shape: (1, 4096)
Dropout output shape: (1, 4096)
Dense output shape: (1, 10)

```

Скачиваем данные [Fashion-MNIST] с маленьким разрешением  $28 \times 28$  пикселей. Но чтобы они подходили под оригинальную AlexNet увеличим их размер до  $224 \times 224$  (аргумент `resize` в функции `d2l.load_data_fashion_mnist`)

Это сделано только для совместимости с оригинальной сетью, на практике вы можете подобрать архитектуру под свои размерности.

```

input_shape=(None,224, 224, 3)
nn=net()
nn.build(input_shape)
nn.summary()

```

➡ Model: "sequential\_1"

| Layer (type)                   | Output Shape        | Param #    |
|--------------------------------|---------------------|------------|
| conv2d_5 (Conv2D)              | (None, 54, 54, 96)  | 34,944     |
| max_pooling2d_3 (MaxPooling2D) | (None, 26, 26, 96)  | 0          |
| conv2d_6 (Conv2D)              | (None, 26, 26, 256) | 614,656    |
| max_pooling2d_4 (MaxPooling2D) | (None, 12, 12, 256) | 0          |
| conv2d_7 (Conv2D)              | (None, 12, 12, 384) | 885,120    |
| conv2d_8 (Conv2D)              | (None, 12, 12, 384) | 1,327,488  |
| conv2d_9 (Conv2D)              | (None, 12, 12, 256) | 884,992    |
| max_pooling2d_5 (MaxPooling2D) | (None, 5, 5, 256)   | 0          |
| flatten_1 (Flatten)            | (None, 6400)        | 0          |
| dense_3 (Dense)                | (None, 4096)        | 26,218,496 |
| dropout_2 (Dropout)            | (None, 4096)        | 0          |
| dense_4 (Dense)                | (None, 4096)        | 16,781,312 |
| dropout_3 (Dropout)            | (None, 4096)        | 0          |
| dense_5 (Dense)                | (None, 10)          | 40,970     |

Total params: 46,787,978 (178.48 MB)

Trainable params: 46,787,978 (178.48 MB)

Non-trainable params: 0 (0.00 B)

batch\_size = 128 # размер пакета

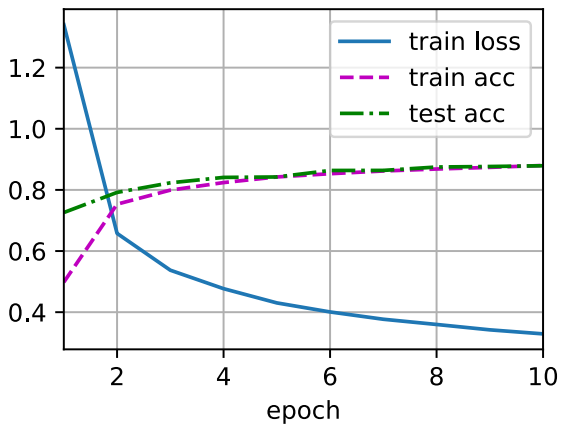
train\_iter, test\_iter = d2l.load\_data\_fashion\_mnist(batch\_size, resize=224) # загружаем и масштабируем

➡ Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz>  
29515/29515 ————— 0s 0us/step  
Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz>  
26421880/26421880 ————— 2s 0us/step  
Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz>  
5148/5148 ————— 0s 1us/step  
Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz>  
4422102/4422102 ————— 1s 0us/step

Обучаем сеть AlexNet на наших данных (10-15 минут).

```
%%time
lr, num_epochs = 0.01, 10 # шаг обучения, число эпох.
d2l.train_ch6(net, train_iter, test_iter, num_epochs, lr, d2l.try_gpu()) # обучаем
```

```
loss 0.329, train acc 0.880, test acc 0.879
1383.0 examples/sec on /GPU:0
CPU times: user 10min 9s, sys: 46.5 s, total: 10min 55s
Wall time: 7min 54s
<Sequential name=sequential_2, built=True>
```



## Задания

1. AlexNet довольно сложная для низкоразмерных Fashion-MNIST данных, упростите модель, но не потеряйте в качестве распознавания.
2. Сделайте то же непосредственно для изображений  $28 \times 28$ .
3. Измените размер пакета batch size и проверьте его влияние на аккуратность ассигасу и занимаемую память GPU.

## Ссылки

Использованы и адаптированы материалы

[https://d2l.ai/chapter\\_convolutional-modern/alexnet.html#](https://d2l.ai/chapter_convolutional-modern/alexnet.html#)